

5-5 · 파티셔닝과 셔플링 최적화 · 정리·회고 0.5H

# 정리·회고 및 산출물 점검

나눠서 하되, 나누는 비용을 안다 — 5주차 마무리

수도권ICT 이노베이션스퀘어  
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

# 30분 뒤에는

---

- ① 5주차 닳새의 배운 것과 남은 결과물을 한 표로 되짚는다
- ② 증상별 튜닝 체크리스트로 어디부터 볼지 안다
- ③ 자주 겪는 오류 네 가지의 원인과 해결을 안다
- ④ 산출물(mart.py·마트 결과·실측표)이 제출 형태인지 점검한다

**Spark 클러스터는 stop 만 — down -v 는 MinIO 볼륨까지 지워 이번 주 테이블이 사라집니다**

# 5주차에 지나온 길

| 차시  | 배운 것                     | 남은 결과물                        |
|-----|--------------------------|-------------------------------|
| 5-1 | 분산 처리 · 지연 평가 · Spark UI | UI 를 읽는 눈                     |
| 5-2 | 마스터-워커 클러스터와 잡 제출        | docker-compose.yml            |
| 5-3 | s3a · Iceberg 연결         | lake.db.orders                |
| 5-4 | DataFrame API 와 SparkSQL | product_sales · daily_revenue |
| 5-5 | 파티셔닝 · 셔플 최적화            | mart.py · daily_sales_mart    |

한 주를 관통한 생각 — 「나눠서 하되, 나누는 비용을 안다」 병렬은 공짜가 아니고 셔플이 그 값이다

# 여덟 개를 스스로 확인해 보세요

---

- 1 파티션이 태스크 병렬성의 단위임을 설명한다
- 2 좁은·넓은 변환을 구분하고 셔플이 생기는 지점을 짚는다
- 3 repartition(셔플 있음)과 coalesce(셔플 없음)의 차이를 실습으로 확인했다
- 4 groupBy 전후로 Stage 가 나뉘고 Shuffle Read/Write 를 UI 에서 봤다
- 5 shuffle.partitions 를 바꿔 실행 시간을 비교했다
- 6 스큐를 만들어 한 태스크만 오래 걸리는 것을 확인했다
- 7 mart.py 를 spark-submit 으로 실행해 마트 테이블을 만들었다
- 8 배치를 두 번 돌려 먹등을 확인하고 산출물을 제출 형태로 정리했다

# 증상 → 먼저 볼 곳 → 흔한 처방

| 증상        | 먼저 볼 곳             | 흔한 처방                 |
|-----------|--------------------|-----------------------|
| 전체가 느리다   | Stage 별 시간         | 가장 긴 Stage 의 Exchange |
| 한 태스크만 오래 | Task 시간 Max/Median | 스큐 — 키 분포 확인          |
| 메모리 부족    | 파티션 수              | repartition 으로 조각을 작게 |
| 작은 파일이 많다 | 저장 직전 파티션 수        | coalesce 로 정리         |
| 조인이 느리다   | 계획의 조인 방식          | 작은 쪽 브로드캐스트           |
| 빈 태스크가 많다 | shuffle.partitions | 데이터 크기에 맞게 줄이기        |

측정 없이 정하지 않는다 — 처음엔 기본값으로 돌리고 UI 를 본다

# 증상으로 찾는 원인과 해결

| 증상                             | 원인                         | 해결                                          |
|--------------------------------|----------------------------|---------------------------------------------|
| 집계 중 익스큐터가<br>OutOfMemoryError | 파티션이 너무 적거나<br>한 키에 몰림(스큐) | repartition 으로 잘게<br>키 분포는 groupBy 로 확인     |
| MinIO 에 자잘한<br>Parquet 수십 개    | 저장 직전 파티션 수 =<br>출력 파일 수   | 저장 전 coalesce(1) 또는<br>repartition(n) 으로 정리 |
| coalesce(16) 인데<br>파티션 수 그대로   | coalesce 는 합치기만<br>늘릴 수 없다 | 늘릴 땐 repartition(16)<br>전체 셔플이니 필요한지 먼저     |
| 큰 결과에 coalesce(1)<br>잡이 죽는다    | 한 태스크가<br>전체를 떠안는다         | repartition(n) 으로 적당히<br>파일당 100~200MB 기준   |

네 가지 모두 「파티션 수」 하나로 통한다 — 너무 적어도, 많아도, 치우쳐도 손해

# 자주 나오는 질문 네 가지

---

## 1 shuffle.partitions 를 항상 줄이면 좋나요?

아니다 — 데이터가 크면 늘려야 한다. AQE 가 켜져 있으면 대개 알아서 조정하니 문제가 실제로 보일 때만

## 2 셔플을 아예 없앨 수는 없나요?

groupBy·join 이 있으면 원리상 필요 — 필터 먼저·열 줄이기·브로드캐스트로 줄이는 것이 현실적 목표

## 3 cache 와 repartition 중 뭘 먼저 하나요?

여러 번 쓸 데이터면 cache, 병렬이 모자라면 repartition — 둘 다 필요하면 repartition 후 cache

## 4 실무에서 파티션 수를 어떻게 정하나요?

기본값으로 돌리고 UI 를 본다 — Task 가 수십 ms 면 줄이고, 메모리가 부족하면 늘린다

6주차로 이어지는 것

# 배치에서 흘러오는 데이터로

1

## 6주차 Kafka — 흘러오는 데이터

파티션 개념이 Kafka 에도 그대로 나온다 — 이름도 같다

2

## Spark Structured Streaming

오늘 배운 DataFrame API 로 스트림을 다룬다 — 문법이 거의 같다

3

## CDC 와 Airflow

6주차 CDC 에 4-3 의 MERGE 와 Spark 가 함께 · 8주차 Airflow 가 mart.py 를 스케줄로 부른다

docker compose 는

stop 만  
(-v 금지)

`cd ~/data-eng/week5 → docker compose stop → mkdir -p ~/data-eng/week6 — 6주차 Kafka 는 메모리를 많이 쓴다`

다음 회차 예고

## 6-1 · 실시간 데이터와 Kafka

### 이론 1H

01

실시간 데이터의 특징  
배치 vs 스트리밍

### 이론·실습 2.5H

02

Kafka 핵심 구조  
CLI 로 첫 송수신

### 정리·회고 0.5H

03

체크리스트 점검  
용어 복습 · 질의응답

준비: Docker Desktop 실행(메모리 8GB 이상 권장) · `docker --version` 확인 · 2주차 MySQL 컨테이너는 6-4 CDC 에서 다시 쓰니 지우지 말 것

5-5 · 정리·회고 0.5H 마무리

## 다음 회차는 6-1 실시간 데이터와 Kafka

5주차까지는 「모아 두었다가 한꺼번에」, 6주차는 「생기는 대로 흘려보내며」 — Kafka 를 처음 띄우고 메시지를 주고받습니다

✓ 배치 vs 스트리밍 — 처리 단위·지연시간

✓ 브로커·토픽·파티션·오프셋

✓ Docker 로 KRaft 단일 브로커 기동

✓ CLI 로 토픽 만들고 메시지 송수신

준비할 것

Docker 8GB 이상 · Spark 는 stop